Reservoir computing

Karol Bednarz

March 24, 2026

Contents

1 Introduction **1**

2 Background **1**

3 Memristor Device Characterization and Modeling **2**

4 Reservoir Computing System Design **2**
4.1 Physical Circuit Implementation 2

5 Experimental Results **4**

6 Discussion & Conclusion **4**

1 Introduction

TODO:

- Motivation: edge AI, neuromorphic computing, in-memory computation
- Limitations of software RC: energy cost, latency
- Memristors as natural reservoir nodes: fading memory, nonlinearity, physical dynamics
- Gap: few hardware RC systems demonstrated end-to-end on classification tasks
- Contribution: physical 8x8 memristor crossbar reservoir + custom MMS model + MNIST classification pipeline
- Paper organization

2 Background

TODO: Reservoir Computing

- RC framework: fixed reservoir, trained readout only
- Key properties required: echo state property, nonlinear kernel expansion
- Why physical substrates are attractive

Memristors in Neuromorphic Systems:

- Memristor fundamentals: pinched hysteresis, state equation
- Prior work: memristor crossbars for synaptic weights, RC, and logic
- Crossbar arrays: passive matrix addressing, sneak path issues

3 Memristor Device Characterization and Modeling

TODO: Measurement Setup:

- Physical device description (material stack, crossbar geometry, e.g. 8x8)
- Characterization: I-V sweeps, pulse response, variability across cells

MMS Model Development:

- Model formulation: state-space representation, nonlinear dynamics
- Parameter extraction: from device characterization data
- Model validation: comparison with experimental results

For the simulation and design of the reservoir computing system, we use Mean Metastable Switch (MMS) memristor model introduced in [2, 1]. The MMS model is a simplified representation of memristor's behavior, focusing on the average switching characteristics rather than the detailed dynamics. It captures essential features of the device by describing transitions between the high-resistance state (HRS) and the low-resistance state (LRS) based on the applied voltage. This model describes memristor's dynamics as

$$\frac{\mathrm{d}x}{\mathrm{d}t} = \frac{1}{\tau}\left(f_{\mathrm{ON}}(v)(1-x) - f_{\mathrm{OFF}}(v)x\right), \tag{1}$$

$$f_{\mathrm{ON}}(v) = \sigma\left(-\beta(v - V_{\mathrm{ON}})\right), \tag{2}$$

$$f_{\mathrm{OFF}}(v) = 1 - \sigma\left(\beta(v - V_{\mathrm{OFF}})\right), \tag{3}$$

where $\sigma(z) = (1 + e^{-z})^{-1}$, $\beta = \frac{q}{kT}$, k is the Boltzmann constant, T is the absolute temperature, q represents the elementary charge, while V_{ON} and V_{OFF} denote the switching voltages to LRS and HRS states, respectively. The instantaneous memductance is defined in (4).

$$\mathcal{G}(x) = \frac{x}{R_{\mathrm{ON}}} + \frac{1-x}{R_{\mathrm{OFF}}}, \tag{4}$$

4 Reservoir Computing System Design

4.1 Physical Circuit Implementation

The proposed physical reservoir computing (RC) system is centered on an 8×8 crossbar array of memristive devices. Within this architecture, each memristor functions as a nonlinear reservoir node, where its conductance state dynamically evolves as a function of the applied voltage and intrinsic device physics.

To facilitate bidirectional communication with the crossbar array, a dedicated peripheral analog-digital interface is employed. The signal pathways and control mechanisms are defined as follows:

- **Input Interfacing:** Input data, such as pixel intensity values from the MNIST dataset, are mapped to proportional analog voltage signals using a DAC856 digital-to-analog converter (DAC). To ensure optimal signal integrity and adequate driving strength, these voltages are subsequently buffered by OPA427 operational amplifiers before being applied to the rows of the crossbar array, systematically stimulating the memristive nodes.

- **Output Readout:** The reservoir's state is evaluated by measuring the output currents from the crossbar columns. These currents are converted into measurable voltages via series-connected resistors. The resulting voltages are buffered by OPA427 operational amplifiers and digitized by an 8-channel 16-bit analog-to-digital converter (ADC) integrated within an STM32H755ZI microcontroller.

- **System Control and State Initialization:** The STM32H755ZI microcontroller serves as the central processing unit, orchestrating the input encoding, temporal synchronization, and readout harvesting. Furthermore, the circuit incorporates analog single-pole single-throw (SPST) switches to systematically reset the memristor conductance states between discrete input samples, ensuring independent transient responses for each inference cycle.

The idealized block diagram of the system is depicted in Figure 1, illustrating the interconnections between the PC, microcontroller, crossbar array, and peripheral components.

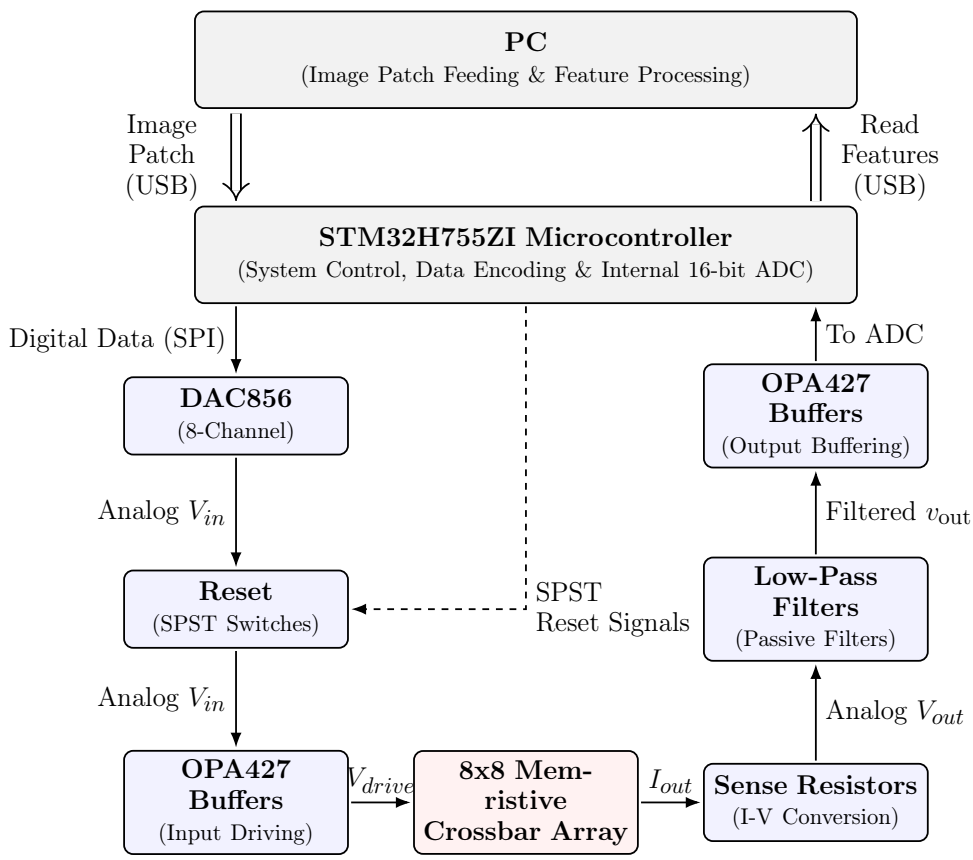

Figure 1: Schematic block diagram of the memristor-based reservoir computing system. The STM32H755ZI microcontroller interfaces with the DAC and ADC to drive the crossbar array and read out the reservoir states, while the PC handles data preprocessing and readout training.

TODO: Crossbar Reservoir Architecture

- Input encoding: voltage mapping of pixel patches / features

- Reservoir topology: how crossbar columns/rows act as nodes
- Readout: linear classifier (ridge regression or similar) on harvested currents

Training Protocol

- Only readout layer trained; reservoir weights fixed (physical)
- Dataset: MNIST preprocessing (patch extraction, normalization)
- Readout training: ridge regression / least-squares

Simulation Framework

- MMS-based SPICE or ODE simulation for design-space exploration
- Correspondence between simulation and hardware measurements

5 Experimental Results

TODO: Device Characterization Results - I-V curves, endurance, variability statistics
MNIST Classification Performance

- Accuracy vs. baseline (software ESN, logistic regression)
- Effect of reservoir size / input encoding on accuracy
- Confusion matrix

Analysis

- Role of memristive nonlinearity and fading memory in classification
- Impact of device-to-device variability (feature vs. bug)

6 Discussion & Conclusion

TODO:

- What the MMS model captures vs. misses
- Scalability: path from 8×8 to larger arrays
- Limitations: sneak paths, write endurance, thermal drift
- Comparison with related hardware RC systems
- Summary of contributions
- Key result (accuracy figure)
- Outlook: larger crossbars, online learning, spike-based input encoding

References

[1] L. Minati et al. "A chaotic circuit based on a physical memristor". In: *Chaos, Solitons & Fractals* 138 (Sept. 2020), p. 109990. ISSN: 0960–0779. DOI: 10.1016/j.chaos.2020.109990.

[2] Timothy W. Molter and M. Alexander Nugent. "The Generalized Metastable Switch Memristor Model". In: *CNNA 2016; 15th Int. Workshop on Cellular Nanoscale Networks and their Applications*. 2016, pp. 1–2.